

# TMM4128:

# Machine Learning for Engineers

**Andrei Lobov**

e-mail: andrei.lobov@ntnu.no

phone: +47 45 913 455

room: 213, Verkstedteknisk

web: <http://www.lobov.biz/academia>

Slides:> <http://lobov.biz/academia/mle/250227>

# Announcements

- First reference group meeting Fri. 28.02.
- For joining a group in A2 (A3) - for group re-assignments contact [andrei.lobov@ntnu.no](mailto:andrei.lobov@ntnu.no)

# Outline : Introduction to Artificial Neural Networks (code [samples](#))

- Definitions / Motivation
- Implementing MLPs with Keras
  - ...
  - Building a Regression MLP Using the Sequential API
  - Building Complex Models Using the Functional API
  - Building Dynamic Models Using the Subclassing API
  - Saving and Restoring a Model
  - Using Callbacks
  - Visualization Using TensorBoard
- Fine-Tuning Neural Network Hyperparameters
  - Number of Hidden Layers
  - Number of Neurons per Hidden Layer
  - Learning Rate, Batch Size and Other Hyperparameters

# Building an Image Classifier Using the Sequential API

*Fashion MNIST*: the images represent fashion items rather than handwritten digits, so each class is more diverse and the problem turns out to be significantly more challenging than MNIST.



Figure 10-11. Samples from Fashion MNIST

# Fashion MNIST

- Using Keras to Load the Dataset
- Creating the Model Using the Sequential API
- Compiling the Model
- Training and Evaluating the Model
- Using the Model to Make Predictions

# Last Friday

- From BNN to ANN.
- Perceptron and MLP, activation functions.
- Start with Keras and TensorFlow

# Building a Regression MLP Using the Sequential API (Steps 48-52)

Building Regression MLP with the Sequential API is similar to what was done for classification: main differences - the output layer has a single neuron. Loss function is the mean squared error.

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

housing = fetch_california_housing()

X_train_full, X_test, y_train_full, y_test = train_test_split(
    housing.data, housing.target)
X_train, X_valid, y_train, y_valid = train_test_split(
    X_train_full, y_train_full)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_valid_scaled = scaler.transform(X_valid)
X_test_scaled = scaler.transform(X_test)
```

```
model = keras.models.Sequential([
    keras.layers.Dense(30, activation="relu", input_shape=X_train.shape[1:]),
    keras.layers.Dense(1)
])
model.compile(loss="mean_squared_error", optimizer="sgd")
history = model.fit(X_train, y_train, epochs=20,
                    validation_data=(X_valid, y_valid))
mse_test = model.evaluate(X_test, y_test)
X_new = X_test[:3] # pretend these are new instances
y_pred = model.predict(X_new)
```

# Building a Regression MLP Using the Sequential API (Steps 48-52)

Sequential API is quite easy to use. These are extremely common, but it is sometimes useful to build neural networks with more complex topologies, or with multiple inputs or outputs.

Thus, the **Functional API**.



# Building Complex Models Using the Functional API (Steps 53-64) (1)

- This architecture makes it possible for the neural network to learn both deep patterns (using the deep path) and simple rules (through the short path).
- A regular MLP forces all the data to flow through the full stack of layers, thus simple patterns in the data may end up being distorted by this sequence of transformations.

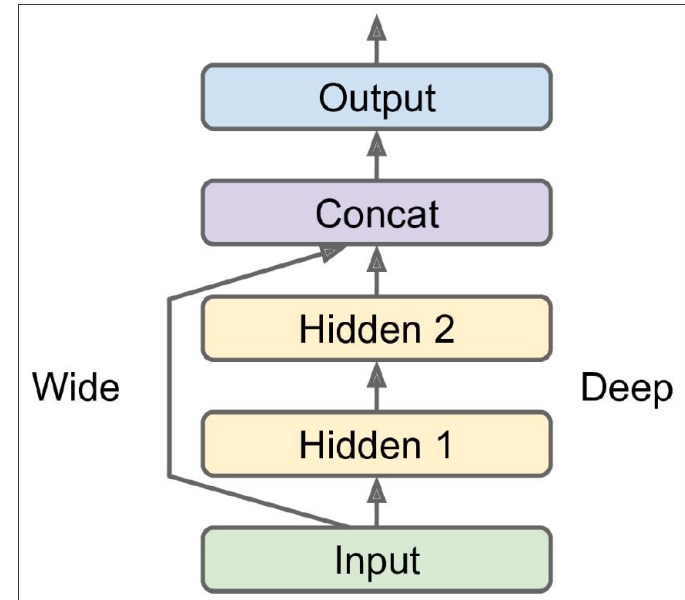


Figure 10-13. Wide and Deep Neural Network

# Building Complex Models Using the Functional API (Steps 53-64) (2)

```
input_ = keras.layers.Input(shape=X_train.shape[1:])
hidden1 = keras.layers.Dense(30, activation="relu")(input_)
hidden2 = keras.layers.Dense(30, activation="relu")(hidden1)
concat = keras.layers.concatenate([input_, hidden2])
output = keras.layers.Dense(1)(concat)
model = keras.models.Model(inputs=[input_], outputs=[output])
```

1. create an Input object. (we may have multiple inputs)
2. create a Dense layer with 30 neurons and using the ReLU activation function. *input* as "function" parameter.
3. create a second hidden layer, and again we use it as a function.
4. create a Concatenate() layer to concatenate the input and the output of the second hidden layer.
5. create a Keras Model, specifying which inputs and outputs to use.

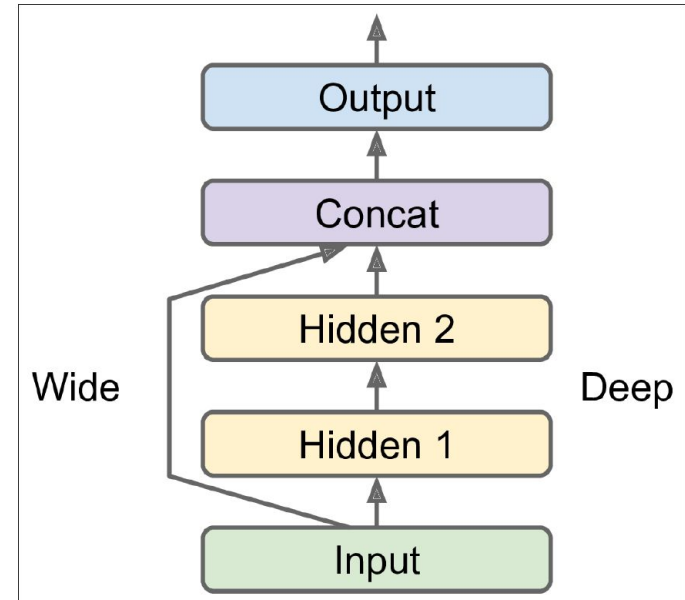


Figure 10-13. Wide and Deep Neural Network

# Building Complex Models Using the Functional API (Steps 53-64) (3)

```
input_A = keras.layers.Input(shape=[5], name="wide_input")
input_B = keras.layers.Input(shape=[6], name="deep_input")
hidden1 = keras.layers.Dense(30, activation="relu")(input_B)
hidden2 = keras.layers.Dense(30, activation="relu")(hidden1)
concat = keras.layers.concatenate([input_A, hidden2])
output = keras.layers.Dense(1, name="output")(concat)
model = keras.models.Model(inputs=[input_A, input_B], outputs=[output])

model.compile(loss="mse", optimizer=keras.optimizers.SGD(learning_rate=1e-3))

X_train_A, X_train_B = X_train[:, :5], X_train[:, 2:]
X_valid_A, X_valid_B = X_valid[:, :5], X_valid[:, 2:]
X_test_A, X_test_B = X_test[:, :5], X_test[:, 2:]
X_new_A, X_new_B = X_test_A[:3], X_test_B[:3]

history = model.fit((X_train_A, X_train_B), y_train, epochs=20,
                    validation_data=((X_valid_A, X_valid_B), y_valid))
mse_test = model.evaluate((X_test_A, X_test_B), y_test)
y_pred = model.predict((X_new_A, X_new_B))
```

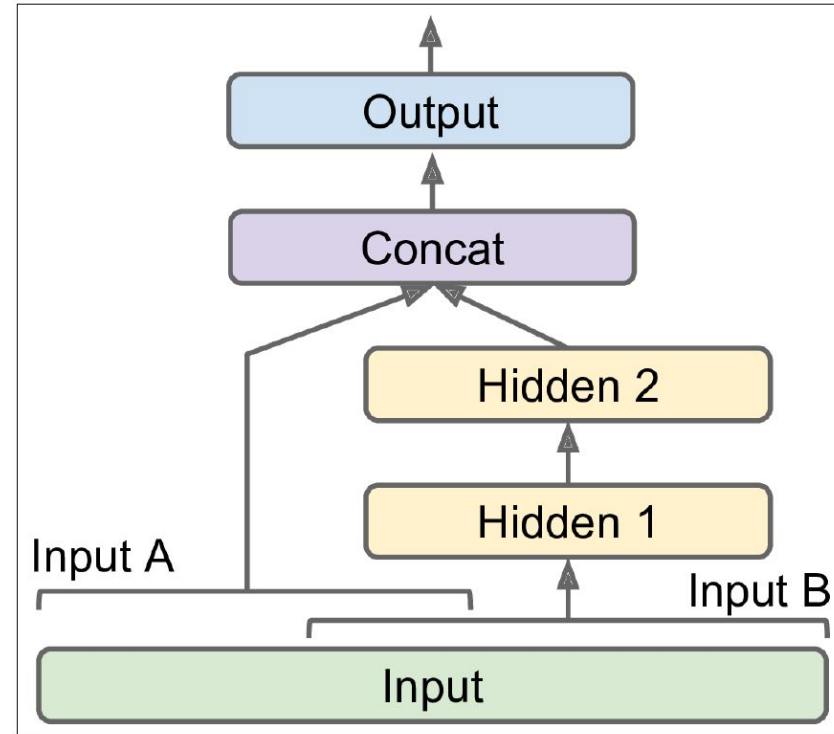


Figure 10-14. Handling Multiple Inputs

# Building Complex Models Using the Functional API (Steps 53-64) : Multiple outputs

```
[...] # Same as above, up to the main output layer
output = keras.layers.Dense(1)(concat)
aux_output = keras.layers.Dense(1)(hidden2)
model = keras.models.Model(inputs=[input_A, input_B],
                           outputs=[output, aux_output])
```

**Task-specific**, e.g. to locate and classify the main object in a picture. This is both a regression task (the coordinates of the center, its width and height) and a classification task.

**Multiple independent tasks** to perform based on the same data. Possible to train one neural network per task, but often one gets better results on all tasks by training a single neural network with one output per task. This is because the neural network can learn features in the data that are useful across tasks.

**Regularization technique** (i.e., a training constraint whose objective is to reduce overfitting and thus improve the model's ability to generalize). To add some auxiliary outputs in a neural network architecture to ensure that the underlying part of the network learns something useful on its own.

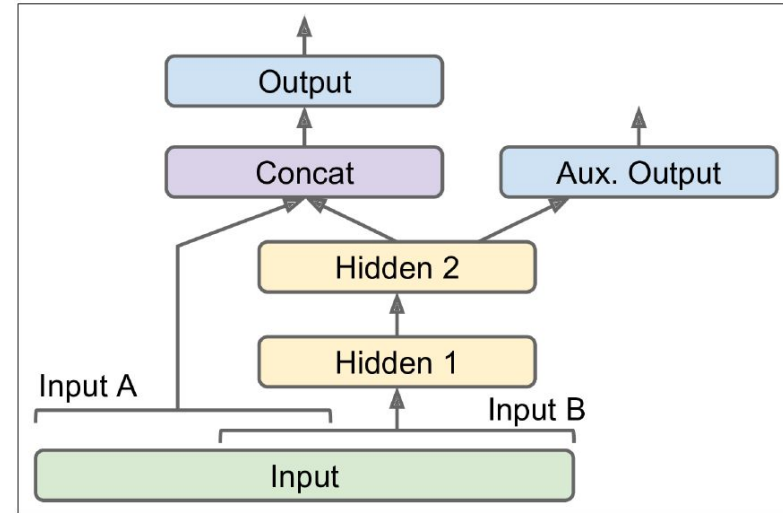


Figure 10-15. Handling Multiple Outputs – Auxiliary Output for Regularization

# Using the Subclassing API to Build Dynamic Models ([Steps 65-66](#))

To potentially vary on shape, dynamic behaviour..., but model's architecture is hidden within the `call()` method.

```
class WideAndDeepModel(keras.models.Model):
    def __init__(self, units=30, activation="relu", **kwargs):
        super().__init__(**kwargs)
        self.hidden1 = keras.layers.Dense(units, activation=activation)
        self.hidden2 = keras.layers.Dense(units, activation=activation)
        self.main_output = keras.layers.Dense(1)
        self.aux_output = keras.layers.Dense(1)

    def call(self, inputs):
        input_A, input_B = inputs
        hidden1 = self.hidden1(input_B)
        hidden2 = self.hidden2(hidden1)
        concat = keras.layers.concatenate([input_A, hidden2])
        main_output = self.main_output(concat)
        aux_output = self.aux_output(hidden2)
        return main_output, aux_output

model = WideAndDeepModel(30, activation="relu")
```

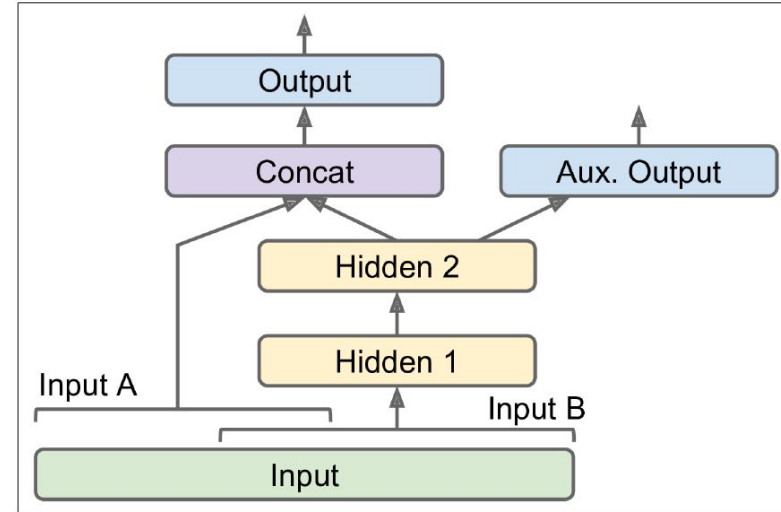


Figure 10-15. Handling Multiple Outputs – Auxiliary Output for Regularization

```
model.compile(loss=["mse", "mse"], loss_weights=[0.9, 0.1], optimizer=keras.optimizers.SGD(learning_rate=1e-3))
history = model.fit((X_train_A, X_train_B), (y_train, y_train), epochs=10,
                    validation_data=((X_valid_A, X_valid_B), (y_valid, y_valid)))
total_loss, main_loss, aux_loss = model.evaluate((X_test_A, X_test_B), (y_test, y_test))
y_pred_main, y_pred_aux = model.predict((X_new_A, X_new_B))
```

# Saving and Restoring a Model (Steps 68-75)

Saving a trained Keras model.

```
model.save("my_keras_model.keras")
```

```
model = keras.models.load_model("my_keras_model.keras")
```

```
model.predict(X_new)
```

# Using Callbacks during Training (Steps 76-81) (1)

## Using checkpoints

```
model.compile(loss="mse", optimizer=keras.optimizers.SGD(learning_rate=1e-3))
checkpoint_cb = keras.callbacks.ModelCheckpoint("my_keras_model.keras", save_best_only=True)
history = model.fit(X_train, y_train, epochs=10,
                    validation_data=(X_valid, y_valid),
                    callbacks=[checkpoint_cb])
model = keras.models.load_model("my_keras_model.keras") # rollback to best model
mse_test = model.evaluate(X_test, y_test)
```



# Using Callbacks during Training (Steps 76-81) (2)

The number of epochs can be set to a large value since training will stop automatically when there is no more progress.

There is no need to restore the best model saved in this case since the EarlyStopping callback will keep track of the best weights and restore them for us at the end of training.

```
early_stopping_cb = keras.callbacks.EarlyStopping(patience=10,  
                                                    restore_best_weights=True)  
history = model.fit(X_train, y_train, epochs=100,  
                    validation_data=(X_valid, y_valid),  
                    callbacks=[checkpoint_cb, early_stopping_cb])
```



# Using Callbacks during Training (Steps 76-81) (3)

## Other possible callbacks to consider:

on\_train\_begin(),  
on\_train\_end(),  
on\_epoch\_begin(),  
on\_batch\_end() .

To use during evaluation and predictions

on\_test\_begin(), on\_test\_end(),  
on\_test\_batch\_begin(), on\_test\_batch\_end()  
on\_predict\_begin(), on\_predict\_end(),  
on\_predict\_batch\_begin(), on\_predict\_batch\_end()

# Using TensorBoard for Visualization

check here [Steps 82-92](#) (2019 ed.) and [Steps 83-92](#) (2022 ed.)

# Visualization Using TensorBoard (1)

- TensorBoard is an interactive visualization tool.
- Can be used to view the learning curves during training, compare learning curves between multiple runs, visualize the computation graph, analyze training statistics, view images generated by your model, visualize complex multidimensional data projected down to 3D and automatically clustered, and more...
- Output the data you want to visualize to special binary log files called *event files*.

# Visualization Using TensorBoard (2)

- Each binary data record is called a *summary*.
- TensorBoard server will monitor the log directory, and it will automatically pick up the changes and update the visualizations:
  - allows you to visualize live data (with a short delay), such as the learning curves during training.
- In general, point the TensorBoard server to a root log directory, and configure the program so that it writes to a different subdirectory every time it runs. This way, the same TensorBoard server instance will allow you to visualize and compare data from multiple runs of your program, without getting everything mixed up.

# Visualization Using TensorBoard (3)

```
root_logdir = os.path.join(os.curdir, "my_logs")

def get_run_logdir():
    import time
    run_id = time.strftime("run_%Y_%m_%d-%H_%M_%S")
    return os.path.join(root_logdir, run_id)

run_logdir = get_run_logdir()
```

```
[...] # Build and compile your model
tensorboard_cb = keras.callbacks.TensorBoard(run_logdir)
history = model.fit(X_train, y_train, epochs=30,
                    validation_data=(X_valid, y_valid),
                    callbacks=[tensorboard_cb])
```

```
my_logs
├── run_2019_01_16-16_51_02
│   └── events.out.tfevents.1547628669.mycomputer.local.v2
└── run_2019_01_16-16_56_50
    └── events.out.tfevents.1547629020.mycomputer.local.v2
```

# Visualization Using TensorBoard :

<http://localhost:6006>

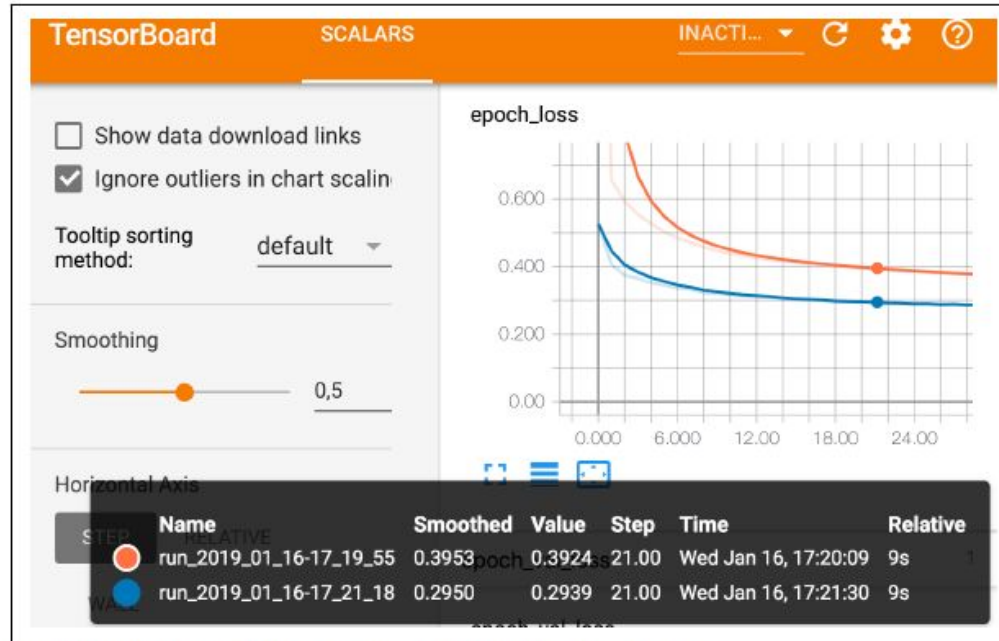


Figure 10-16. Visualizing Learning Curves with TensorBoard

# Fine-Tuning Neural Network Hyperparameters

Number of Hidden Layers

Number of Neurons per Hidden Layer

Learning Rate, Batch Size and Other Hyperparameters

# Fine-Tuning Neural Network Hyperparameters (1)

The flexibility of neural networks is also one of their main drawbacks: there are **many hyperparameters to tweak**. Not only can you use any imaginable network architecture, but even in a simple MLP you can change the number of layers, the number of neurons per layer, the type of activation function to use in each layer, the weight initialization logic, and much more.

So, how do you know what combination of hyperparameters is the best for your task?



# Fine-Tuning Neural Network Hyperparameters (2)

One option is to try many combinations of hyperparameters and see which one works best on the validation set (or using K-fold cross-validation). For this, one approach is use [GridSearchCV](#) or [RandomizedSearchCV](#) to explore the hyperparameter space.

Step 1: a function that will build and compile a Keras model, given a set of hyperparameters:

```
def build_model(n_hidden=1, n_neurons=30, learning_rate=3e-3, input_shape=[8]):  
    model = keras.models.Sequential()  
    options = {"input_shape": input_shape}  
    for layer in range(n_hidden):  
        model.add(keras.layers.Dense(n_neurons, activation="relu", **options))  
        options = {}  
    model.add(keras.layers.Dense(1, **options))  
    optimizer = keras.optimizers.SGD(learning_rate)  
    model.compile(loss="mse", optimizer=optimizer)  
    return model
```

# Fine-Tuning Neural Network Hyperparameters (3)

Next, create a KerasRegressor based on this build\_model() function:

```
from scikeras.wrappers import KerasRegressor  
keras_reg = KerasRegressor(build_model)
```

The KerasRegressor object is a thin wrapper around the Keras model built using build\_model(). Since we did not specify any hyperparameter when creating it, it will just use the default hyperparameters we defined in build\_model().

We can use this object like a regular Scikit-Learn regressor train it using its fit(), then evaluate it using its score(), and to make predictions use its predict() method.

Any extra parameter you pass to the fit() method **will simply get passed to the underlying Keras model.**

# Fine-Tuning Neural Network Hyperparameters (4)

```
keras_reg.fit(X_train, y_train, epochs=100,  
              validation_data=(X_valid, y_valid),  
              callbacks=[keras.callbacks.EarlyStopping(patience=10)])  
mse_test = keras_reg.score(X_test, y_test)  
y_pred = keras_reg.predict(X_new)
```

We don't want to train and evaluate a single model, but to train hundreds of variants and see which one performs best on the validation set.

Since **there are many hyperparameters**, it is preferable to use a **randomized search** rather than **grid search**.

# Fine-Tuning Neural Network Hyperparameters (5)

```
from scipy.stats import reciprocal
from sklearn.model_selection import RandomizedSearchCV

param_distributions = {
    "n_hidden": [0, 1, 2, 3],
    "n_neurons": np.arange(1, 100),
    "learning_rate": reciprocal(3e-4, 3e-2),
}

rnd_search_cv = RandomizedSearchCV(keras_reg, param_distributions, n_iter=10, cv=3)
rnd_search_cv.fit(X_train, y_train, epochs=100,
                  validation_data=(X_valid, y_valid),
                  callbacks=[keras.callbacks.EarlyStopping(patience=10)])

>>> rnd_search_cv.best_params_
{'learning_rate': 0.0033625641252688094, 'n_hidden': 2, 'n_neurons': 42}
>>> rnd_search_cv.best_score_
-0.3189529188278931
>>> model = rnd_search_cv.best_estimator_.model
```

# Fine-Tuning Neural Network Hyperparameters (6)

Other ways to search than randomly:

- [Hyperopt](#): a Python library for optimizing over all sorts of complex search spaces (including real values such as the learning rate, or discrete values such as the number of layers).
- [Hyperas](#), [kopt](#) or [Talos](#): optimizing hyperparameters for Keras model (the first two are based on Hyperopt).
- [Scikit-Optimize](#) (skopt): a general-purpose optimization library. The BayesSearchCV class performs Bayesian optimization using an interface similar to GridSearchCV.
- [Sklearn-Deap](#): a hyperparameter optimization library based on evolutionary algorithms, also with a GridSearchCV-like interface.

# Number of Hidden Layers (1)

- In many cases, you can just begin with a single hidden layer and you will get reasonable results.
- It has been shown that an MLP with just one hidden layer can **model even the most complex functions provided it has enough neurons**.
- For a long time, these facts convinced researchers that there was no need to investigate any deeper neural networks.
- But they overlooked the fact that **deep networks have a much higher parameter efficiency than shallow ones**: they can model complex functions using **exponentially fewer neurons than shallow nets**, allowing them to reach much better performance with the same amount of training data.

## Number of Hidden Layers (2)

- Not only does this hierarchical architecture help DNNs converge faster to a good solution, it also improves their ability to generalize to new datasets.
- For example, if you have already trained a model to recognize faces in pictures, and you now want to train a new neural network to recognize hairstyles, then you can kickstart training by reusing the lower layers of the first network. Instead of randomly initializing the weights and biases of the first few layers of the new neural network, you can initialize them to the value of the weights and biases of the lower layers of the first network.
- This way the network will not have to learn from scratch all the low-level structures that occur in most pictures; it will only have **to learn the higher-level structures** (e.g., hairstyles). This is called transfer learning.

# Number of Hidden Layers (3)

- For many problems, one can start with just one or two hidden layers and it will work just fine (e.g., you can easily reach above 97% accuracy on the MNIST dataset using just one hidden layer with a few hundred neurons, and above 98% accuracy using two hidden layers with the same total amount of neurons, in roughly the same amount of training time).
- For more complex problems, you can **gradually ramp up the number of hidden layers, until you start overfitting the training set**. Very complex tasks, such as large image classification or speech recognition, typically require networks with dozens of layers (or even hundreds, but not fully connected ones).



# Number of Neurons per Hidden Layer (1)

- The number of neurons in the input and output layers is determined by the type of input and output your task requires. For example, MNIST task requires  $28 \times 28 = 784$  input neurons and 10 output neurons.
- Hidden layers, it used to be a common practice to size them to form a **pyramid**, with fewer and fewer neurons at each layer—the rationale being that many low level features can coalesce into far fewer high-level features.
  - For example, a typical neural network for MNIST may have three hidden layers, the first with 300 neurons, the second with 200, and the third with 100.
- However, this practice has been largely abandoned now, as it seems that simply using the same number of neurons in all hidden layers performs just as well in most cases, or even better, and there is just one hyperparameter to tune instead of one per layer—for example, all hidden layers could simply have 150 neurons. However, depending on the dataset, it can sometimes help to make the first hidden layer bigger than the others.

## Number of Neurons per Hidden Layer (2)

- Just like for the number of layers, you can **try increasing the number of neurons gradually until the network starts overfitting**. In general you will get a better performance by increasing the number of layers than the number of neurons per layer.
- Unfortunately, finding the perfect amount of neurons is still somewhat of a dark art. A simpler approach is to pick a model with more layers and neurons than you actually need, then use early stopping to prevent it from overfitting.

# Learning Rate, Batch Size and Other Hyperparameters (1)

The number of hidden layers and neurons are not the only hyperparameters.

- The learning rate. In general, the optimal learning rate is about half of the maximum learning rate. A simple approach for tuning the learning rate is to start with a large value that makes the training algorithm diverge, then divide this value by 3 and try again, and repeat until the training algorithm stops diverging. At that point, you generally won't be too far from the optimal learning rate.
- Choosing a better optimizer than plain old Mini-batch Gradient Descent (and tuning its hyperparameters) is also quite important.

# Learning Rate, Batch Size and Other Hyperparameters (2)

- The batch size. In general the optimal batch size will be lower than 32.
- Choosing a better optimizer than plain old Mini-batch Gradient Descent (and tuning its hyperparameters) is also important.
- Choice of the activation function: in general, the ReLU activation function is a good default for all hidden layers. For the output layer, it really depends on the task.
- In most cases, the number of training iterations does not actually need to be tweaked: just use early stopping instead.

See also the following.

# Takeaways

- Ways to setup ANNs/MLPs of different topologies in Keras.
- Factors affecting fine-tuning of ANNs.

## A2: Unsupervised learning on time series data

You are to implement a solution for anomalies detection in time series data. This can be used to monitor industrial equipment, detect anomalies and call human operators for checking the situation closer. Take a time series dataset made for any **operating machine / equipment**. Check if and which anomalies can be detected in the dataset.

Follow the ML project checklist. **Evaluate and report on each phase of the checklist. Compare performance** of different unsupervised learning techniques. Why do some perform better than others? How would you suggest to use the solution in online mode, i.e. getting real-time data from a machine?

Check for datasets here:

\* <https://www.kaggle.com/>